

Zero to Hero Study Handbook: Transaction Anomaly Detection (Unsupervised)

Module 1: Foundations & Architecture

1.1 What this project does

This repository is a learning-first anomaly detection system for payment transactions. It ranks transactions from most suspicious to least suspicious using unsupervised models, then measures ranking quality against fraud labels when labels exist.

Main use cases in this repo: - Learn how to build an unsupervised anomaly pipeline on highly imbalanced fraud data. - Compare three detectors under one scoring convention (**higher score = more anomalous**). - Produce a ranked alert queue from a CSV using a CLI (`src/score.py`).

Core references: - `README.md` - `src/data.py` - `src/features.py` - `src/models.py` - `src/metrics.py` - `src/score.py` - `build_notebook.py`

1.2 Core paradigms and patterns used here

Definitions first: - Unsupervised anomaly detection: learn what “normal” looks like without using class labels for training. - Ranking-based evaluation: evaluate whether high anomaly scores surface real fraud cases near the top. - Functional pipeline style: most logic is in stateless functions that transform data and return outputs. - Strategy/fallback loading pattern: data source selection tries multiple sources in order until one succeeds. - Unified scoring convention: model outputs are normalized so all detectors share the same interpretation.

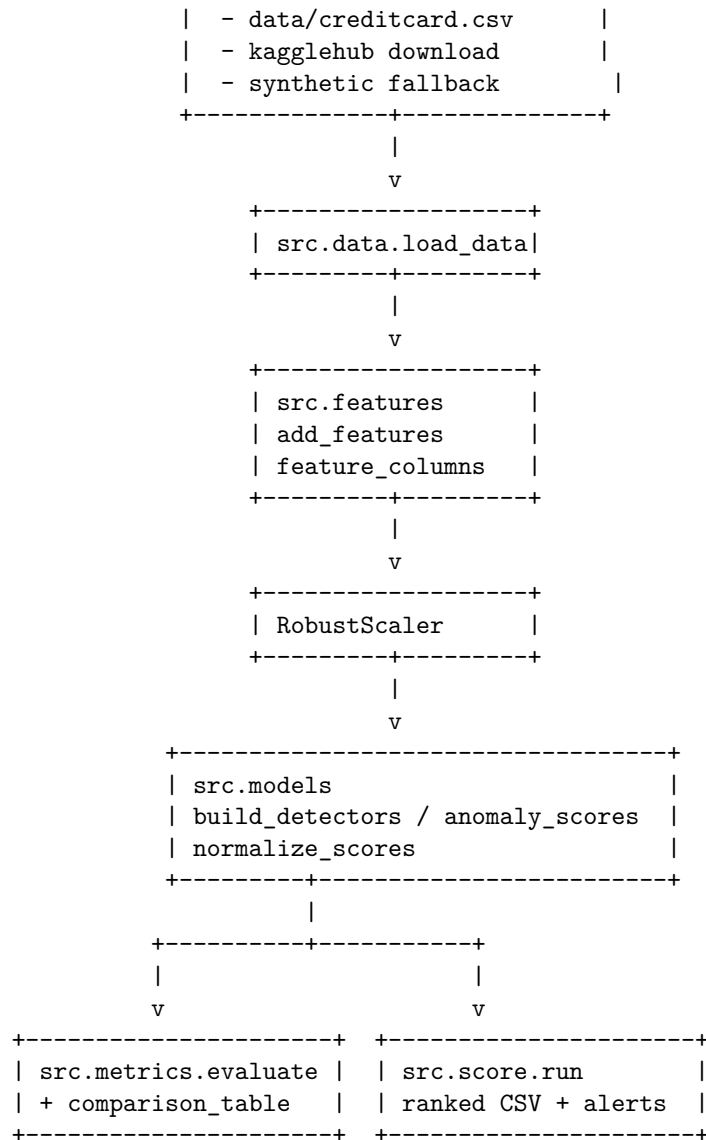
How these appear in code: - Functional modules: `add_features`, `feature_columns`, `anomaly_scores`, `evaluate`, `comparison_table`. - Fallback chain: `load_data()` in `src/data.py` checks local file, then Kaggle, then synthetic generator. - Shared model contract: `build_detectors()` returns detector objects plus metadata (`Detector` dataclass). - Comparable scores: `normalize_scores()` rank-normalizes model outputs to `[0, 1]`.

1.3 Architecture and component interaction

High-level component map: - Data ingestion: `src/data.py` - Feature engineering: `src/features.py` - Modeling/scoring: `src/models.py` - Evaluation metrics: `src/metrics.py` - Batch ranking CLI: `src/score.py` - Notebook source-of-truth builder: `build_notebook.py` - Learning notebook artifact: `transaction_anomaly_detection.ipynb`

Main architecture flow (text diagram):

```
+-----+
| transaction CSV source |
```



Design intent visible in files: - `README.md` frames labels as a “yardstick”, not model training signal. - `src/models.py` enforces one score direction despite estimator API differences. - `src/score.py` operationalizes the recommended detector (Isolation Forest) for batch output.

Module 2: Repository Map

Important-first map for new contributors:

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
README.md	Project purpose, dataset context, setup/run commands, baseline findings	N/A	Kaggle env vars (KAGGLE_USERNAME, KAGGLE_KEY), example CLI flags (--top-k, --contamination)
pyproject.toml	Package metadata and dependency declarations	N/A	requires-python = ">=3.12.10", dependencies list (scikit-learn, pandas, kagglehub, jupyter, etc.)
.python-version	Local Python version pin	N/A	3.12.10
src/__init__.py	Package-level constants and module summary	constant SEED	SEED = 42
src/data.py	Data loading and fallback strategy	_load_local, _load_kaggle, make_synthetic, load_data	PROJECT_ROOT, DATA_DIR, LOCAL_CSV, prefer_kaggle
src/features.py	Feature engineering over transaction stream	RAW_V, add_features, feature_columns	window sizes (60, 600), rolling window 100
src/models.py	Detector construction and scoring normalization	Detector dataclass, build_detectors, anomaly_scores, normalize_scores	detector keys (IsolationForest, LocalOutlierFactor, OneClassSVM), contamination, seed
src/metrics.py	Ranking-oriented evaluation	precision_recall_at_k, evaluate, comparison_table	metric keys (PR_AUC, ROC_AUC, Precision@K, Recall@K)
src/score.py	Batch scoring endpoint and CSV writer	run, main	CLI flags (--input, --output, --top-k, --contamination)

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>build_notebook.py</code>	Deterministically builds <code>transaction_anomaly_detection</code> from code-defined cells	<code>md</code> , <code>code</code> helpers and notebook assignment	<code>kernelspec</code> name <code>python3</code> , language version <code>3.12.10</code>
<code>transaction_anomaly_detection.ipynb</code>	Main detection tutorial notebook artifact	Notebook cells generated by <code>build_notebook.py</code>	Uses imports from <code>src/*</code> , seed and model flow
<code>uv.lock</code>	Resolved dependency lockfile for reproducible installs	N/A	Exact dependency versions for <code>uv sync</code>
<code>.gitignore</code>	Ignore generated assets and local artifacts	N/A	ignores <code>data/</code> , <code>outputs/</code> , <code>*.csv</code> , <code>.venv</code> , model artifacts

Recommended first-read sequence: 1. `README.md` 2. `src/score.py` 3. `src/data.py` 4. `src/features.py` 5. `src/models.py` 6. `src/metrics.py` 7. `build_notebook.py`

Module 3: Core Execution Flows

3.1 Core data contract (input schema and key shapes)

Canonical transaction schema used across modules: - Required for full fidelity: `Time`, `V1..V28`, `Amount` - Optional but used for evaluation/reporting: `Class`

Source evidence: - `make_synthetic()` in `src/data.py` constructs columns in this exact order: `["Time", "V1"... "V28", "Amount", "Class"]` - `feature_columns()` in `src/features.py` pulls available `V1..V28` plus engineered columns.

Engineered columns added by `add_features()`: - `Amount_log` - `Hour` - `Hour_sin` - `Hour_cos` - `txn_count_60s` - `txn_count_600s` - `amount_roll_mean` - `amount_z`

Model input matrix: - `X`: robust-scaled numeric array from `feat[cols]` where `cols = feature_columns(feat)`.

Output shapes: - `load_data(...)` -> `tuple[pd.DataFrame, str]` where `str` is one of `local`, `kaggle`, `synthetic`. - `build_detectors(...)` -> `dict` mapping model keys to (`estimator`, `Detector`). - `evaluate(...)` -> `dict` with:

- PR_AUC: float - ROC_AUC: float - Precision@{k}: float - Recall@{k}: float - src.score.run(...) -> pd.DataFrame containing original columns
 plus: - anomaly_score (float rank-normalized to [0, 1]) - alert (integer 0/1; top-k rows flagged)

3.2 Flow A: Data loading with fallback chain

Entrypoint: load_data(prefer_kaggle: bool = True) in src/data.py.

Step-by-step: 1. Try local CSV: _load_local() reads data/creditcard.csv if it exists. 2. If not found and prefer_kaggle=True, try _load_kaggle(): - imports kagglehub - calls kagglehub.dataset_download("mlg-ulb/creditcardfraud") - copies downloaded creditcard.csv into data/creditcard.csv 3. If both fail, return make_synthetic() output and source label synthetic.

Code fragment:

```
def load_data(prefer_kaggle: bool = True) -> tuple[pd.DataFrame, str]:
    df = _load_local()
    if df is not None:
        return df, "local"
    if prefer_kaggle:
        df = _load_kaggle()
        if df is not None:
            return df, "kaggle"
    return make_synthetic(), "synthetic"
```

3.3 Flow B: Feature engineering and model-ready columns

Entrypoints: - add_features(df) in src/features.py - feature_columns(df) in src/features.py

Step-by-step: 1. Copy input DataFrame. 2. Transform amount (Amount_log = log1p(Amount)). 3. Derive hour-of-day from Time, then cyclic encodings (Hour_sin, Hour_cos). 4. Compute stream velocity features via trailing window counts (60s, 600s) using np.searchsorted. 5. Compute rolling amount baseline and z-score (amount_roll_mean, amount_z) over last 100 transactions. 6. Select final model feature names: - Present raw PCA columns V1..V28 - Plus engineered feature names that exist - Excludes Time, Amount, Hour, Class

3.4 Flow C: Detector scoring and metric computation

Entrypoints: - build_detectors(contamination=0.01, seed=SEED) in src/models.py - anomaly_scores(key, estimator, X) in src/models.py - normalize_scores(scores) in src/models.py - evaluate(y_true, scores, k=None) in src/metrics.py

Step-by-step: 1. Build three detectors: - Isolation Forest - Local Outlier Factor (novelty=False) - One-Class SVM approximation (Nystroem

+ `SGDOneClassSVM`) 2. Score anomaly magnitude so that larger is always more anomalous: - LOF: `-negative_outlier_factor_` - Others: `-decision_function(X)` 3. Rank-normalize each model score to `[0, 1]` for cross-model comparability. 4. Evaluate ranking quality with labels: - PR-AUC - ROC-AUC - Precision@K - Recall@K

Key convention enforced in code: - `src/models.py` docstring and `anomaly_scores()` ensure one global score direction.

3.5 Flow D: Batch scoring CLI (`src/score.py`)

Entrypoint: module execution via `python -m src.score`.

CLI arguments (`main()`): - `--input` (optional CSV path; default `None`) - `--output` (default `outputs/ranked_transactions.csv`) - `--top-k` (default 100) - `--contamination` (default 0.01)

`run(input_path, output_path, top_k, contamination)` sequence: 1. Load data from CSV if `--input` is given; otherwise call `load_data()`. 2. Add engineered features and select model columns. 3. Scale with `RobustScaler`. 4. Build only `IsolationForest` from `build_detectors(...)`. 5. Compute and normalize anomaly scores. 6. Sort descending by score. 7. Add `alert` column and set first `top_k` rows to 1. 8. Write output CSV and print summary; if `Class` exists, print `precision@K`.

Code fragment:

```
df["anomaly_score"] = normalize_scores(scores)
df = df.sort_values("anomaly_score", ascending=False).reset_index(drop=True)
df["alert"] = 0
df.loc[: top_k - 1, "alert"] = 1
df.to_csv(output_path, index=False)
```

3.6 Flow E: Notebook generation path (`build_notebook.py`)

What it does: - Creates a notebook object (`nbf.v4.new_notebook()`). - Appends markdown/code cells with helper functions `md(...)` and `code(...)`. - Writes `transaction_anomaly_detection.ipynb`.

Why it matters: - The notebook is generated from a plain Python source file, so tutorial logic is reviewable in code review. - Notebook metadata is set explicitly: - kernelspec name `python3` - language version `3.12.10`

Module 4: Setup & Run Guide

4.1 Prerequisites on a clean machine

From repository files: - Python requirement: `>=3.12.10` (`pyproject.toml`)
- Local pin: `3.12.10` (`.python-version`) - Package manager/workflow: `uv`
(`README.md` commands + `uv.lock`)

4.2 Installation steps

```
git clone https://github.com/pypi-ahmad/transaction-anomaly-detection.git
cd transaction-anomaly-detection
uv sync
```

What this sets up: - Creates `.venv` - Installs dependencies pinned by `uv.lock`

4.3 Environment variables and data access

Optional Kaggle credentials (only needed if local `data/creditcard.csv` is absent and Kaggle download is attempted):

```
export KAGGLE_USERNAME=<your-username>
export KAGGLE_KEY=<your-key>
```

Code-level credential note in `src/data.py` docstring: - Supports environment credentials and `~/.kaggle/kaggle.json`.

4.4 Typical command sequences

Main notebook:

```
uv run jupyter lab transaction_anomaly_detection.ipynb
```

Regenerate notebook from source script:

```
uv run python build_notebook.py
```

Headless execute notebook:

```
uv run jupyter nbconvert --to notebook --execute --inplace \
  --ExecutePreprocessor.kernel_name=python3 \
  transaction_anomaly_detection.ipynb
```

Batch scoring CLI:

```
uv run python -m src.score --input data/creditcard.csv \
  --output outputs/ranked.csv --top-k 100
```

4.5 External services, migrations, and seeding

- Database migrations: none in this repository.
- Service bootstrap scripts: none in this repository.

- Data seeding: handled by fallback synthetic generator (`make_synthetic`) when real data is unavailable.
-

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for new learners

1. Start with `README.md` to understand objective, dataset, and intended outputs.
2. Read `src/__init__.py` to note reproducibility seed (`SEED = 42`).
3. Read `src/data.py` to understand the three-source data loading strategy.
4. Read `src/features.py` to learn engineered signal construction and feature selection.
5. Read `src/models.py` to understand detector setup and anomaly score direction normalization.
6. Read `src/metrics.py` to understand ranking-focused evaluation under class imbalance.
7. Read `src/score.py` to connect everything into an operational scoring pipeline.
8. Read `build_notebook.py` to see how the end-to-end narrative and experiments are assembled.
9. Inspect `transaction_anomaly_detection.ipynb` as the final learner-facing artifact.

5.2 Practice exercises (with targeted file reading)

1. Exercise: Trace exact fallback behavior when no local CSV exists.
 - Read: `src/data.py`
 - Task: Explain every branch from `load_data(prefer_kaggle=True)` to final return value.
2. Exercise: Reconstruct full engineered feature list and why each exists.
 - Read: `src/features.py`
 - Task: List all added columns and identify which ones encode time cyclicity vs stream velocity.
3. Exercise: Explain why `Amount` is replaced by `Amount_log` in model inputs.
 - Read: `src/features.py` docstring and `feature_columns()`
 - Task: State which raw columns are excluded and why.
4. Exercise: Compare model API handling differences in `anomaly_scores()`.
 - Read: `src/models.py`
 - Task: Explain why LOF uses `negative_outlier_factor_` while other models use `decision_function`.
5. Exercise: Show how cross-model comparability is achieved.
 - Read: `src/models.py`
 - Task: Explain `normalize_scores()` and why rank normalization is used instead of min-max on raw outputs.

6. Exercise: Explain metrics choice under extreme class imbalance.
 - Read: `src/metrics.py` and `README.md`
 - Task: Justify why PR-AUC is treated as the headline metric in this project.
7. Exercise: Describe the exact output columns of the batch scoring CLI.
 - Read: `src/score.py`
 - Task: Identify new columns added to output CSV and how alert rows are selected.
8. Exercise: Map notebook build-time vs run-time responsibilities.
 - Read: `build_notebook.py`, `transaction_anomaly_detection.ipynb`
 - Task: Explain what is fixed at notebook construction time and what is computed at notebook execution time.

5.3 Solution outlines

1. Fallback behavior:
 - `load_data` first calls `_load_local`; if found returns `(df, "local")`.
 - If not found and `prefer_kaggle` is true, `_load_kaggle` tries `kagglehub.dataset_download("mlg-ulb/creditcardfraud")`; on success returns `(df, "kaggle")`.
 - Any failure path ends at synthetic: `(make_synthetic(), "synthetic")`.
2. Engineered features:
 - Added: `Amount_log`, `Hour`, `Hour_sin`, `Hour_cos`, `txn_count_60s`, `txn_count_600s`, `amount_roll_mean`, `amount_z`.
 - Cyclic time: `Hour_sin`, `Hour_cos`.
 - Velocity proxies: `txn_count_60s`, `txn_count_600s`.
3. Amount handling:
 - `feature_columns()` excludes raw `Amount` and includes `Amount_log`.
 - Reason from docstring: amount is heavy-tailed; `log1p` stabilizes scale.
4. Model scoring API differences:
 - LOF (`novelty=False`) exposes `negative_outlier_factor_` only after fitting on in-sample data.
 - IsolationForest and One-Class SVM path use `decision_function`, which is “higher = more normal”, so values are negated.
5. Cross-model comparability:
 - `normalize_scores` assigns rank positions and divides by `n-1`.
 - This avoids direct comparison of incompatible raw score scales from different detectors.
6. Metric choice:
 - `README.md` and `src/metrics.py` state ROC-AUC can look optimistic with rare positives.
 - PR-AUC better reflects effectiveness at surfacing fraud in the top-ranked alerts.
7. CLI output columns:
 - Original columns are preserved.

- Added columns: `anomaly_score` (normalized score) and `alert` (top-k rows set to 1).
 - Rows are sorted descending by `anomaly_score`.
8. Notebook build vs run:
- Build-time (`build_notebook.py`): cell content, structure, metadata, and writing `.ipynb`.
 - Run-time (`transaction_anomaly_detection.ipynb` execution): data loading, feature creation, model fitting, scoring, charts, and metric tables.
-

Understanding Verification Checklist

Use this checklist to validate mastery:

- Can you explain the exact fallback order in `load_data()` and when each source label appears?
- Can you list every engineered feature from `add_features()` and the signal it captures?
- Can you explain why `feature_columns()` excludes `Amount`, `Time`, `Hour`, and `Class`?
- Can you describe how each detector is constructed in `build_detectors()` and one tradeoff per model?
- Can you explain why anomaly scores are negated for some estimators and rank-normalized afterward?
- Can you reproduce what `evaluate()` returns, including dynamic key names like `Precision@{k}`?
- Can you walk through `src.score.run()` end-to-end, including CSV input/output transformations?
- Can you explain the difference between notebook source generation (`build_notebook.py`) and notebook execution (`transaction_anomaly_detection.ipynb`)?
- Can you state all setup prerequisites (`uv`, Python version, optional Kaggle env vars) from repository config files?